

光线追踪大作业实验报告

基础光线生成与 BVH 加速渲染

姓名：_____ 学号：_____

2026 年 6 月 14 日

摘要

本次大作业基于课程提供的 C++ 光线追踪框架, 完成了一个 Whitted 风格的光线追踪渲染器。具体实现了四个核心函数: `Renderer::Render` (主光线生成)、`Triangle::getIntersection` (光线-三角形求交, Möller-Trumbore 算法)、`Bounds3::IntersectP` (光线-包围盒求交, slab 测试) 与 `BVHAccel::getIntersection` (BVH 层次遍历加速求交); 并额外完成了 `Scene::castRay_noBVH` 中的完整着色模型 (加分项)。在 1280×960 分辨率、含 **4968** 个三角形的 Stanford Bunny 场景下, BVH 模式约 **0.45 s** 即可渲染出正确的着色图像。本文逐一说明各函数的算法原理、关键代码与实验结果。

1 概述

光线追踪通过**逆向**追踪从相机出发、穿过每个像素射入场景的光线, 求出光线与场景的最近交点并在该点着色, 从而生成图像。当场景中三角形数量较大时, ”对每条光线遍历所有三角形”的暴力做法效率极低; 为此需要使用 **层次包围盒 (Bounding Volume Hierarchy, BVH)** 对求交过程进行加速: 先用廉价的光线-包围盒测试快速排除大片不可能相交的空间, 再递归深入到真正可能相交的少数三角形。整体渲染流程如图 1 所示。

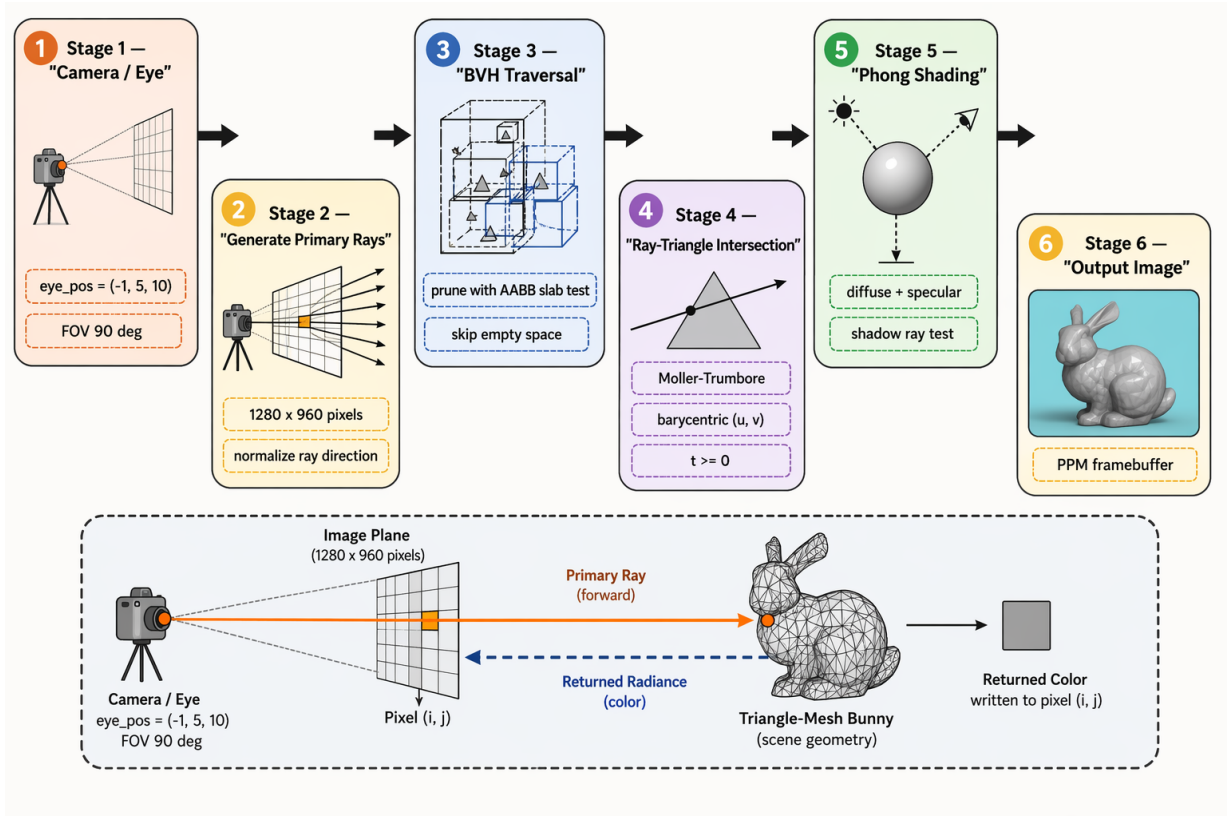


图 1: 光线追踪渲染流程: 相机生成主光线 → BVH 遍历剪枝 → Möller-Trumbore 光线-三角形求交 → Phong 着色 (含阴影测试) → 写入帧缓冲并输出 PPM 图像。

本框架需要在以下文件中补全代码: `Renderer.cpp`、`Triangle.hpp`、`Bounds3.hpp`、`BVH.cpp`，并可选地在 `Scene.cpp` 中补全着色模型。下面分别说明。

2 各函数的实现

2.1 主光线生成: `Renderer::Render`

该函数遍历图像的每个像素 (i, j) ，构造一条从相机出发、穿过该像素中心的光线。设视场角为 fov ，则 $\text{scale} = \tan(\text{fov}/2)$ ，图像宽高比 $\text{aspect} = \text{width}/\text{height}$ 。将像素坐标映射到位于 $z = -1$ 平面上的相机空间坐标：

$$x = \left(2 \cdot \frac{i+0.5}{\text{width}} - 1\right) \cdot \text{aspect} \cdot \text{scale}, \quad (1)$$

$$y = \left(1 - 2 \cdot \frac{j+0.5}{\text{height}}\right) \cdot \text{scale}. \quad (2)$$

其中 $+0.5$ 使光线穿过像素中心， y 方向取负是因为图像行号自上而下增大而相机 y 轴向上。随后归一化方向并投射光线：

```
1 float x = (2.0f * (i + 0.5f) / scene.width - 1.0f) * imageAspectRatio * scale;
2 float y = (1.0f - 2.0f * (j + 0.5f) / scene.height) * scale;
3 Vector3f dir = normalize(Vector3f(x, y, -1));
4 Ray ray(eye_pos, dir);
5 framebuffer[m++] = check_mode ? scene.castRay_noBVH(ray, 0) : scene.castRay(ray, 0);
```

最终把浮点颜色 clamp 到 $[0, 1]$ 并按 P6 格式写出 `binary.ppm`。

2.2 光线-三角形求交: `Triangle::getIntersection` (Möller-Trumbore)

光线 $\mathbf{O} + t\mathbf{D}$ 与三角形 $(\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2)$ 的交点可用重心坐标 (u, v) 表示:

$$\mathbf{O} + t\mathbf{D} = (1 - u - v)\mathbf{P}_0 + u\mathbf{P}_1 + v\mathbf{P}_2. \quad (3)$$

令 $\mathbf{E}_1 = \mathbf{P}_1 - \mathbf{P}_0$, $\mathbf{E}_2 = \mathbf{P}_2 - \mathbf{P}_0$, $\mathbf{T} = \mathbf{O} - \mathbf{P}_0$, 由 Cramer 法则 (Möller-Trumbore) 可得

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\mathbf{E}_1 \cdot (\mathbf{D} \times \mathbf{E}_2)} \begin{bmatrix} \mathbf{E}_2 \cdot (\mathbf{T} \times \mathbf{E}_1) \\ \mathbf{T} \cdot (\mathbf{D} \times \mathbf{E}_2) \\ \mathbf{D} \cdot (\mathbf{T} \times \mathbf{E}_1) \end{bmatrix}. \quad (4)$$

有效交点必须满足 $u \geq 0$, $v \geq 0$, $u + v \leq 1$ (落在三角形内) 且 $t \geq 0$ (位于光线前方)。实现上:

```
1 // 背面剔除: 只对正对相机的面求交, 避免封闭网格内壁干扰
2 if (dotProduct(ray.direction, normal) > 0) return inter;
3
4 Vector3f pvec = crossProduct(ray.direction, e2);
5 float det = dotProduct(e1, pvec);
6 if (fabs(det) < EPSILON) return inter; // 光线与三角形平行
7
8 float invDet = 1.0f / det;
9 Vector3f tvec = ray.origin - v0;
10 float u = dotProduct(tvec, pvec) * invDet;
11 if (u < 0 || u > 1) return inter;
12 Vector3f qvec = crossProduct(tvec, e1);
13 float v = dotProduct(ray.direction, qvec) * invDet;
14 if (v < 0 || u + v > 1) return inter;
15 float t = dotProduct(e2, qvec) * invDet;
16 if (t < 0) return inter; // 交点在光线起点之后
17
18 inter.happened = true; inter.coords = ray(t);
19 inter.normal = normal; inter.distance = t;
20 inter.obj = this; inter.m = m;
```

实现要点: 相比框架自带的辅助函数, 这里在函数内部直接实现了 Möller-Trumbore, 并补充了两处健壮性判断——平行判定使用 $|det| < \varepsilon$ (而非严格等于 0), 以及 $t \geq 0$ 剔除位于相机背后的交点, 避免错误交点污染 BVH 的最近距离比较。

2.3 光线-包围盒求交: `Bounds3::IntersectP` (slab 测试)

轴对齐包围盒 (AABB) 可视为三对相互垂直的平行平面 (slab)。光线进入第 k 个 slab 的参数为 $t_k^{\min} = (p_k^{\min} - O_k)/D_k$, 离开为 $t_k^{\max} = (p_k^{\max} - O_k)/D_k$ 。光线进入整个盒子的时刻为各轴进入时刻的最大值, 离开盒子的时刻为各轴离开时刻的最小值:

$$t_{\text{enter}} = \max_k t_k^{\min}, \quad t_{\text{exit}} = \min_k t_k^{\max}. \quad (5)$$


```

4         int(ray.direction.z > 0) };
5 if (!node->bounds.IntersectP(ray, invDir, dirIsNeg)) return Intersection(); // 剪枝
6 if (node->left == nullptr && node->right == nullptr)
7     return node->object->getIntersection(ray); // 叶节点
8 Intersection hit1 = getIntersection(node->left, ray);
9 Intersection hit2 = getIntersection(node->right, ray);
10 return hit1.distance < hit2.distance ? hit1 : hit2; // 取更近者

```

由于 Intersection 的默认 distance 为 double 最大值，未命中的子树自然不会赢得最近距离比较，因此无需额外的命中判断即可正确合并左右结果。

2.5 额外内容：castRay_noBVH 的完整着色

框架在检测模式 (./RayTracing check) 下不会调用 buildBVH(), 故 scene.bvh 为空，原 castRay_noBVH 仅返回纯白以验证求交是否正确。本部分按文档“额外内容”要求，参照上方的 castRay 补全了完整的 Whitted 着色模型。为不依赖场景级 BVH，新增了暴力最近交点函数：

```

1 Intersection Scene::intersect_noBVH(const Ray &ray) const {
2     Intersection nearest; // 默认 distance = DBL_MAX
3     for (uint32_t k = 0; k < objects.size(); ++k) {
4         Intersection it = objects[k]->getIntersection(ray);
5         if (it.happened && it.distance < nearest.distance) nearest = it;
6     }
7     return nearest;
8 }

```

castRay_noBVH 在此基础上完成与 castRay 一致的三分支着色：(1) 反射 + 折射材质用 Fresnel 方程混合反/折射颜色；(2) 镜面反射材质追投反射光线；(3) 漫反射/光泽材质使用 Phong 模型 (漫反射 + 高光)，并对每个光源投射阴影光线判断遮挡：

$$L = \sum_{\text{lights}} (1 - \text{inShadow}) I (\mathbf{N} \cdot \mathbf{l}) k_d c_d + I \max(0, -\mathbf{r} \cdot \mathbf{d})^p k_s, \quad (6)$$

其中阴影测试与递归光线均改用 intersect_noBVH。Bunny 材质为漫反射/光泽，故走 Phong 分支，最终检测模式得到与 BVH 模式逐像素完全一致的着色图像。

3 实验结果

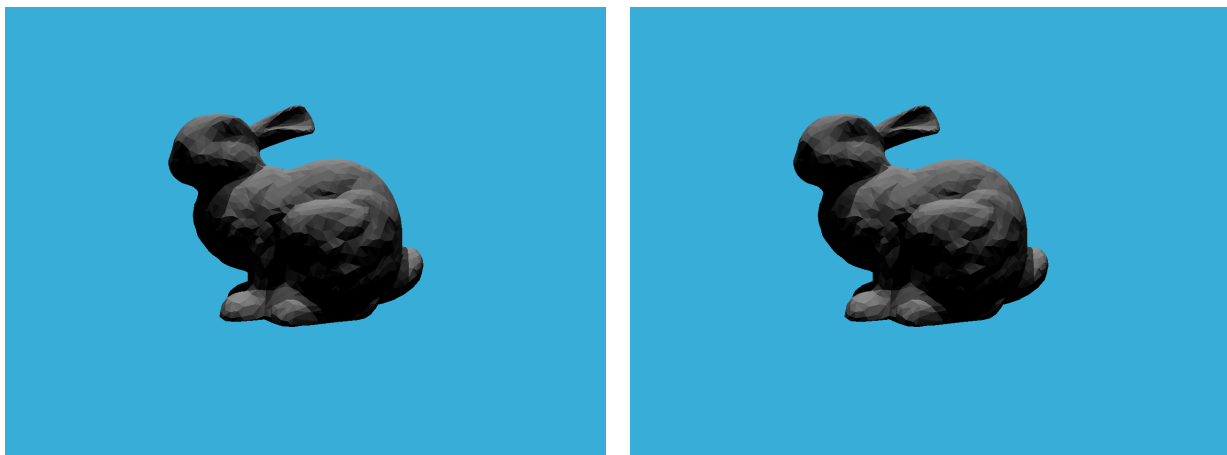
编译与运行：

```

mkdir build && cd build && cmake .. -DCMAKE_BUILD_TYPE=Release && make
./RayTracing # BVH 加速渲染 -> binary.ppm
./RayTracing check # 检测/着色模式 -> binary.ppm

```

两种模式的渲染结果如图 3。BVH 模式 (左) 正确渲染出带漫反射与高光、含自阴影的灰色兔子；检测模式 (右) 在补全着色模型后得到与之一致的图像，验证了求交与着色逻辑的正确性。



(a) BVH 加速渲染 (./RayTracing)

(b) 检测/着色模式 (./RayTracing check)

图 3: 1280×960 、4968 三角形的 Stanford Bunny 渲染结果。两图逐像素一致。

本机 (release 构建) 实测渲染耗时如表 1。

表 1: 渲染耗时 (4968 三角形, 1280×960 , 3 次取稳定值)

模式	是否使用场景级 BVH	渲染耗时
./RayTracing	是	≈ 0.45 s
./RayTracing check	否 (暴力遍历场景物体)	≈ 0.46 s

两者耗时接近, 是因为本场景仅含一个网格物体, 而 `MeshTriangle` 在构造时已对其内部 4968 个三角形建立了一层 BVH——即便关闭**场景级** BVH, 网格**内部**的 BVH 仍在发挥加速作用。作为对照, 文档给出的”完全不使用任何 BVH、逐三角形暴力求交”的参考耗时约为 7 分钟, 而启用 BVH 后降至秒级, 凸显了层次包围盒带来的数量级加速。

4 总结

本次作业完整实现了基于 BVH 加速的 Whitted 风格光线追踪器: 从主光线生成、Möller-Trumbore 光线-三角形求交、AABB slab 包围盒测试, 到 BVH 递归遍历, 并额外补全了非 BVH 路径下的完整着色模型。在四千余三角形的 Bunny 场景上以约 0.45 s 渲染出正确结果。通过本次实践, 加深了对光线生成、解析法求交、空间数据结构加速以及局部光照 (Phong + 阴影) 等图形学核心概念的理解。